

OOP Lab 7: Reading and Practicing the SOLID Principles

Object-Oriented Programming Laboratory

1. Goals

After this lab, you should be able to:

- Read and summarize the SOLID principles from a reference.
- Explain each SOLID principle in simple words.
- Read small pieces of code and find where SOLID is violated.
- Refactor (rewrite) the code into a cleaner design.

2. Pre-lab Reading & Report on SOLID

2.1. Reading Task

Go to the following GitHub repository:

- <https://github.com/alejoalvarez/oop>

Then:

- a) Find and carefully read the section about **SOLID**.
- b) If needed, search for extra explanations on SOLID from other sources.
- c) Do **not** copy-paste directly. You must understand and write in your own words.

2.2. Short Written Report (Summary)

Write a short report (about 0.5–1 page) answering these questions:

1. What is the goal of the SOLID principles? (Why do we use them in OOP?)
2. For each principle, write in your own words:
 - Name of the principle (full name + abbreviation).
 - One or two sentences describing the idea.
 - A very simple example (real life or code idea) that shows the principle.
3. Among the five principles, which one do you think is *most important* for beginners, and why?

You may use the following template in your report (fill in with your own words):

- **S – Single Responsibility Principle (SRP):**
(Your explanation, 1–2 sentences)
Simple example: (Your example)
- **O – Open/Closed Principle (OCP):**
(Your explanation, 1–2 sentences)
Simple example: (Your example)
- **L – Liskov Substitution Principle (LSP):**
(Your explanation, 1–2 sentences)
Simple example: (Your example)
- **I – Interface Segregation Principle (ISP):**
(Your explanation, 1–2 sentences)
Simple example: (Your example)
- **D – Dependency Inversion Principle (DIP):**
(Your explanation, 1–2 sentences)
Simple example: (Your example)

You will submit this report together with the coding exercises.

3. Exercise 1 – Single Responsibility Principle (SRP)

Consider the following class:

```
package srp;

public class StudentManager {

    public void addStudent(String name, String email) {
        // 1. validate data
        if (name == null || name.isBlank()
            || email == null || !email.contains("@")) {
            System.out.println("Invalid data!");
            return;
        }

        // 2. save to "database"
        System.out.println("Saving " + name + " to database...");

        // 3. send welcome email
        System.out.println("Sending welcome email to " + email);
    }
}
```

Tasks

- In a comment inside the code, clearly mark the 3 different responsibilities.
- Create three new classes:

- StudentValidator with a method boolean isValid(String name, String email).
 - StudentRepository with a method void save(String name, String email) (just print to console).
 - EmailService with a method void sendWelcomeEmail(String email) (just print to console).
- c) Rewrite StudentManager so that it **uses** these three classes (composition) instead of doing everything by itself.
- d) Write a small main method to test:
- ```
public static void main(String[] args) {
 StudentValidator validator = new StudentValidator();
 StudentRepository repo = new StudentRepository();
 EmailService emailService = new EmailService();

 StudentManager manager = new StudentManager(validator, repo, emailService);
 manager.addStudent("Tin", "tin@example.com");
}
```
- e) In a short comment or note, explain why the new design follows SRP.

## 4. Exercise 2 – Open/Closed Principle (OCP)

We want to calculate the area of different shapes.

```
package ocp;

public class AreaCalculator {

 public double area(String shape, double size) {
 if (shape.equals("CIRCLE")) {
 return 3.14 * size * size;
 } else if (shape.equals("SQUARE")) {
 return size * size;
 } else {
 return 0;
 }
 }
}
```

### Problems

Every time we add a new shape, we must change the AreaCalculator class. This breaks OCP.

### Tasks

- a) Create an interface:

```
public interface Shape {
 double area();
}
```

b) Create two classes:

- Circle implements Shape and has a radius.
- Square implements Shape and has a side.

c) Rewrite AreaCalculator:

```
public class AreaCalculator {

 public double totalArea(Shape[] shapes) {
 double total = 0;
 for (Shape s : shapes) {
 total += s.area();
 }
 return total;
 }
}
```

d) In main, create an array of shapes (circle + square), and print the total area.

e) Now add a new class Triangle (base, height) that also implements Shape, **without** changing the code inside AreaCalculator. Show in your report that this follows OCP.

## 5. Exercise 3 – Liskov Substitution Principle (LSP) (Conceptual)

Look at this example (do not implement if you do not have time; focus on thinking):

```
public class Bird {
 public void fly() {
 System.out.println("I am flying!");
 }
}

public class Penguin extends Bird {
 @Override
 public void fly() {
 throw new UnsupportedOperationException("Penguins cannot fly");
 }
}
```

### Tasks

- Explain why Penguin is a problem for LSP.  
(Hint: code that uses a Bird and calls fly() will break when we pass a Penguin.)
- Suggest a better design in words or simple class diagrams. For example:
  - Have a base class Bird without fly().
  - Use interfaces like FlyingBird and SwimmingBird.
- (Optional) Implement your improved design in code.

## 6. Exercise 4 – Interface Segregation Principle (ISP)

Consider this interface:

```
public interface Machine {
 void print(String content);
 void scan(String content);
 void fax(String content);
}
```

Now we have a simple printer:

```
public class OldPrinter implements Machine {

 @Override
 public void print(String content) {
 System.out.println("Printing: " + content);
 }

 @Override
 public void scan(String content) {
 // not supported
 throw new UnsupportedOperationException();
 }

 @Override
 public void fax(String content) {
 // not supported
 throw new UnsupportedOperationException();
 }
}
```

### Tasks

- Explain (1–2 sentences) why this breaks ISP.
- Split the big interface into smaller ones, for example:

```
public interface Printer {
 void print(String content);
}

public interface Scanner {
 void scan(String content);
}

public interface Fax {
 void fax(String content);
}
```

- Make:

- OldPrinter implement only Printer.
- (Optional) MultiFunctionMachine implements Printer, Scanner, and Fax.

## 7. Exercise 5 – Dependency Inversion Principle (DIP)

We want to send a welcome message. Here is the original code:

```
public class EmailService {

 public void sendEmail(String to, String message) {
 System.out.println("Sending EMAIL to " + to + ": " + message);
 }
}

public class NotificationService {

 private final EmailService emailService = new EmailService();

 public void sendWelcome(String email) {
 emailService.sendEmail(email, "Welcome to our course!");
 }
}
```

### Problems

- NotificationService depends directly on a concrete class EmailService.
- It is hard to change to SMS or push notification.

### Tasks

a) Create an interface:

```
public interface MessageSender {
 void send(String to, String message);
}
```

b) Make EmailService implement MessageSender:

```
public class EmailService implements MessageSender {
 @Override
 public void send(String to, String message) {
 System.out.println("Sending EMAIL to " + to + ": " + message);
 }
}
```

c) Create another implementation SmsService:

```
public class SmsService implements MessageSender {
 @Override
 public void send(String to, String message) {
 System.out.println("Sending SMS to " + to + ": " + message);
 }
}
```

d) Rewrite NotificationService to depend on the interface:

```

public class NotificationService {

 private final MessageSender sender;

 public NotificationService(MessageSender sender) {
 this.sender = sender;
 }

 public void sendWelcome(String to) {
 sender.send(to, "Welcome to our course!");
 }
}

```

e) In main, create:

```

public static void main(String[] args) {
 NotificationService emailNotif =
 new NotificationService(new EmailService());
 emailNotif.sendWelcome("tin@example.com");

 NotificationService smsNotif =
 new NotificationService(new SmsService());
 smsNotif.sendWelcome("0123456789");
}

```

f) In 1–2 sentences, explain how this follows DIP.

## 8. What to Submit

Each student should submit:

- **Report (PDF, 0.5–1 page)** summarizing the SOLID principles based on the GitHub reference and your own understanding. Also, the explanation for each exercise (1 - 5).
- **Source code** for all exercises (srp, ocp, lsp, isp, dip packages).
- (Optional) Any class diagrams or notes you used to reason about the design.